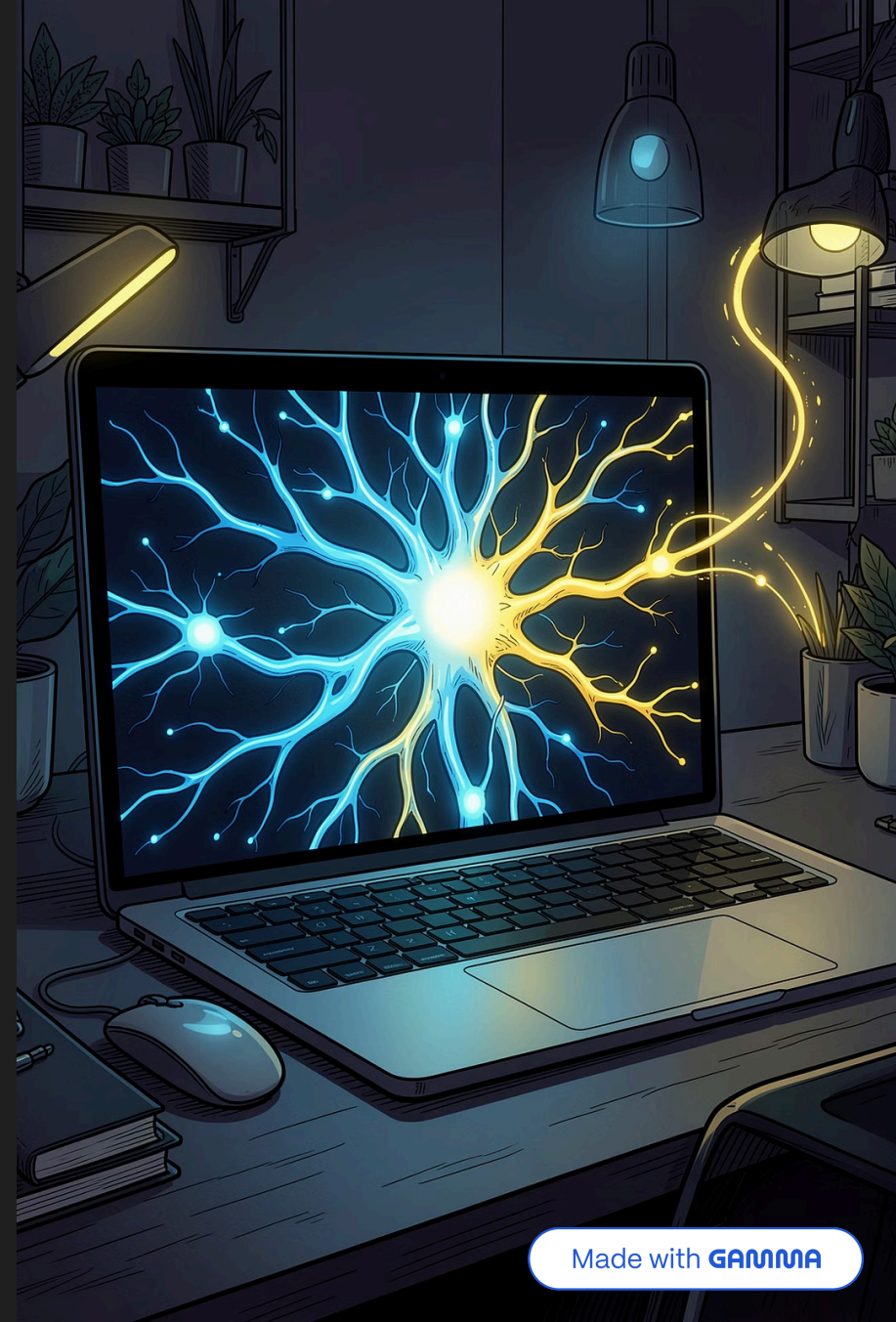


Real-Time Machine Learning on the Web

Teachable Machine Web Integration — Seamless browser-based AI classification, low-latency inference, and a responsive UI built for the modern web.

PRESENTER: ASHARIB

[GITHUB.COM/M-ASHARIB/TEACHABLE_MACHINE](https://github.com/m-asharib/teachable_machine)





Democratizing AI via the Browser

The Goal

Bridge the gap between complex ML models and intuitive user experiences by running classification entirely on the client side.

Core Value

Eliminates server dependencies, keeps data private, and delivers instant visual feedback — all within the browser.

Who It's For

Developers, educators, and product teams seeking rapid prototyping of computer vision or audio recognition.

Modern, Optimized & Scalable Stack



Core ML Engine

Google Teachable Machine models powered by **TensorFlow.js** (@tensorflow/tfjs) — running inference directly in the browser.

Frontend Architecture

Component-based, fast-rendering framework handling real-time video and audio streams without blocking the main thread.

UI & Deployment

Clean, tech-forward aesthetic with responsive layouts. Zero server overhead — fully client-side distributed assets.

Interactive Intelligence at Your Fingertips



Live Capture

Secure browser API access to webcam and microphone with instant toggle states and permission handling.



Multi-Class Detection

Support for custom training classes — gestures, objects, or states — each with individual confidence bars.



Real-Time Classification

High-frequency rendering loops update prediction probabilities dynamically, frame by frame.



Offline-Ready

Once assets load, the ML model runs entirely without internet access — ideal for privacy-first environments.

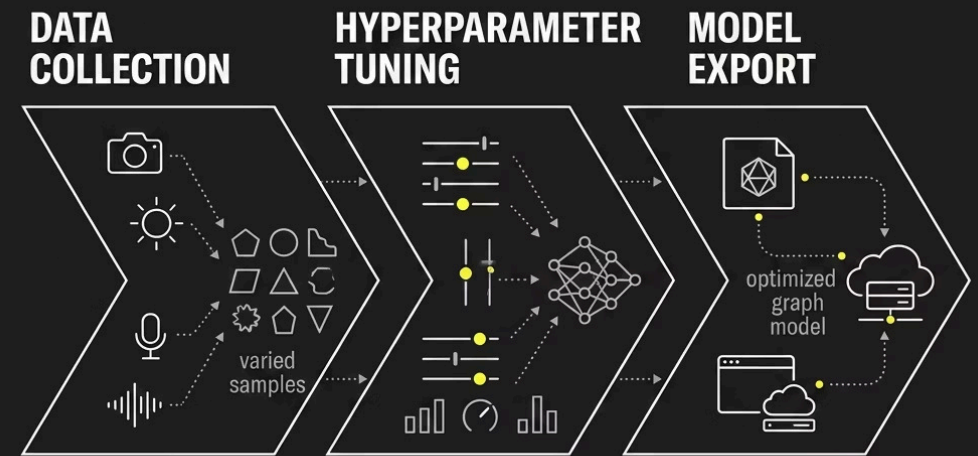
From Raw Input to Trained Model

Data Ingestion

Capturing diverse samples — varied angles, lighting conditions, and background noise — to prevent overfitting and improve generalization.

Export Strategy

Hosted or local execution graph models optimized for web performance and fast initialization.



Intuitive, Bento-Grid Inspired Interface

1

Permission & Initialization

Smooth onboarding with camera/microphone access requests and graceful fallback UI.

2

Stream Visualization

Dedicated viewport displaying live video canvas overlaid with real-time analytics.

3

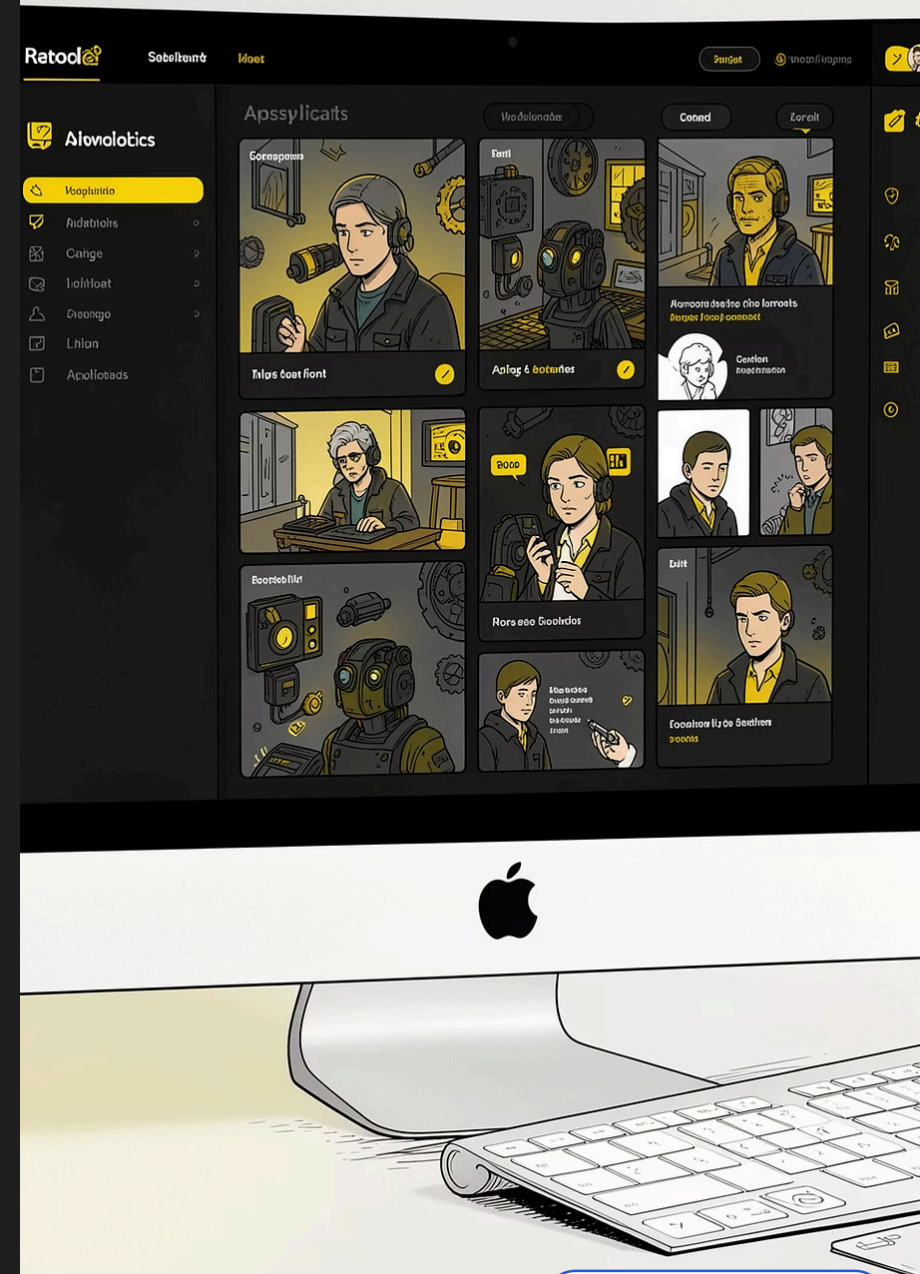
Prediction Dashboard

Animated progress bars displaying confidence intervals from 0% to 100%.

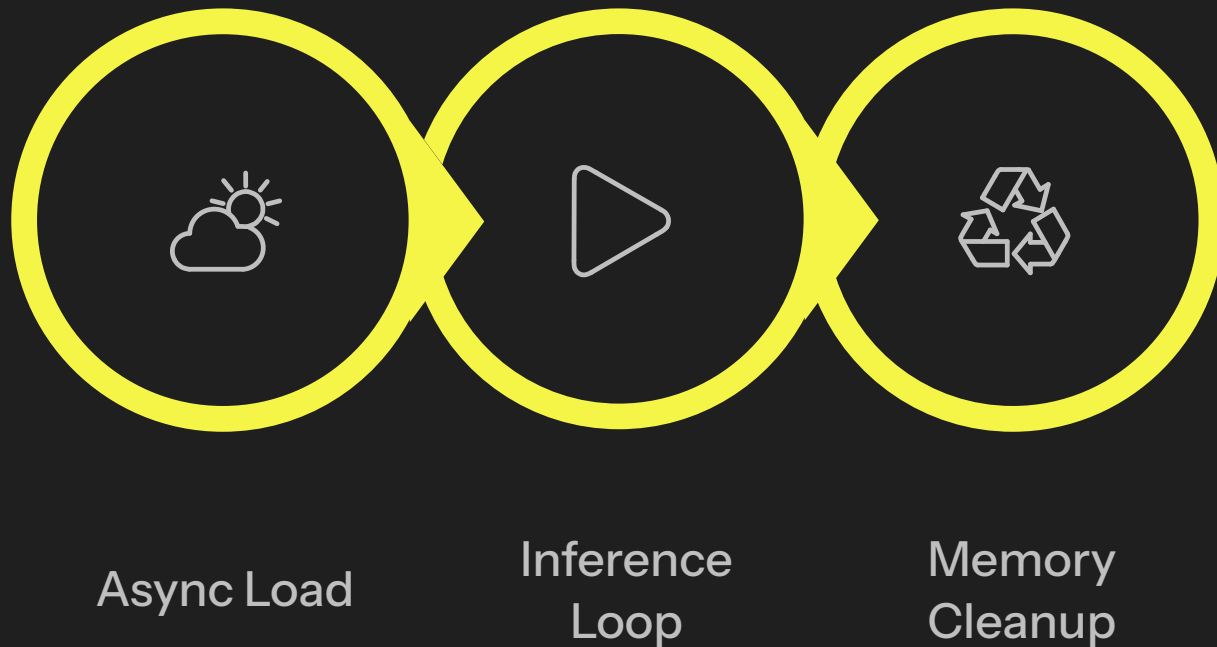
4

Responsive Layout

Adaptive scaling across desktop, tablet, and mobile browsers.



How the Magic Happens Under the Hood



The inference pipeline is engineered for smooth, non-blocking performance — model loading happens asynchronously, predictions run at a steady 60fps via `requestAnimationFrame`, and WebGL resources are explicitly disposed to prevent browser memory leaks.

Overcoming Web-Based Constraints



Challenge: CPU Overload

High CPU usage and frame drops on lower-end devices during live rendering.

Solution: Downscaled input canvas dimensions before passing frames to the TensorFlow layer.



Challenge: Environmental Noise

Varied lighting and background conditions degrading prediction reliability.

Solution: Implemented probability thresholds — actions only trigger when confidence exceeds 85%.

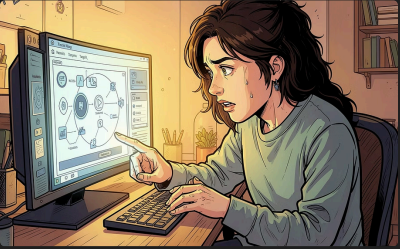


Challenge: Browser Compatibility

Asynchronous media device locks and inconsistent API support across browsers.

Solution: Added fallback error boundaries and precise try-catch loops for media streaming.

Real-World Potential



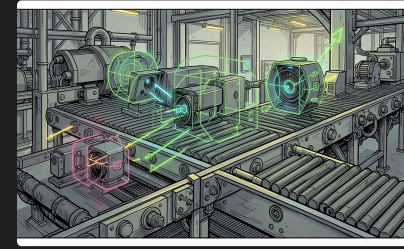
Accessibility Controls

Navigate interfaces using facial expressions, head movements, or voice triggers — enabling hands-free interaction for all users.



Interactive EdTech

Gamified learning applications where students interact with physical cards or objects, bridging the digital and physical worlds.



Industrial Prototyping

Rapid, low-cost proof of concept for sorting lines, security detection, or presence checking — no custom hardware required.



Continuous Evolution

→ Dynamic Transfer Learning

Allow users to append new classes directly from the deployed UI — no retraining pipeline required.

→ Advanced Animation States

Smooth micro-interactions triggered by specific inference outcomes, making feedback feel alive.

→ Mobile-First Optimization

Leverage WebNN and specialized mobile WebGL contexts for maximum battery efficiency on handheld devices.



Key Takeaway: Advanced AI doesn't require massive server backends. This project is a blueprint for privacy-first, low-latency, browser-native machine learning.